YAGNI ("You Ain't Gonna Need It") principle. This is fine if the application never scales. But if it does, the refactoring work will be extensive.

Alternatively, you could invest in a highly scalable architecture from the word go. This would avoid rework, but it would be wasted effort if volume remains low.

A third possibility would be to buy an option to scale up later. You wouldn't implement a fully scalable architecture immediately, but you could spend a little time upfront to see what would be needed to make the initial implementation easily scalable in the future if required. If you don't need to scale, you've only invested a little upfront design time, and if you do, you'll be able to do so at reduced costs.

4.3.2 Options expire

You can't keep an option open forever. In software development, an option expires (that is, you can no longer use it) when you no longer have time to implement it before the related feature is due to be delivered. For example, in figure 4.18, you have the choice between two implementations (solution A and solution B). At this point, you don't know which solution would be best, so you add a layer of code to make it possible to switch to either solution A or B at a later date.

If you decide for solution A, it will take 10 days to integrate. Implementing solution B, on the other hand, would only take 5 days. In practical terms, this means that if you decide to implement solution A, you must do so at least 10 days before the delivery date, which is when your option on solution A expires. If you delay any further, you won't be able to exercise this option. You have a bit more time to opt for solution B, as this option only expires 5 days before the delivery date.

Unlike financial options, you sometimes have the power to push back expiry dates. For example, if you can find a way to integrate solution A more quickly, you can leave that option open longer.

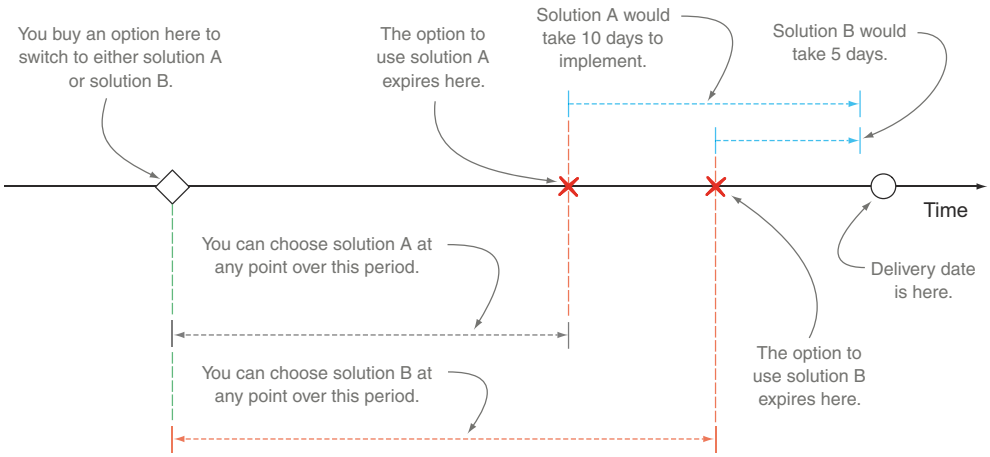


Figure 4.18 Real options expire. Once you pass an option's expiry date, you can no longer exercise this option.